Python NumPy & Pandas: Extended Practice Assignment

Dataset: Student Life & Daily Habits (student_life_dataset_200.csv)

This extended practice workbook provides an additional set of challenges to deepen students' understanding of data wrangling, advanced filtering, basic cleaning, and matrix mathematics using Pandas and NumPy. It continues to build on 'student_life_dataset_200.csv'.

Part 4: Data Sorting & Cleaning (Pandas)

Q11. Sort the dataset to find the top 5 students with the highest GPA. If there is a tie, sort them by their study hours in descending order.

```
# Sorting by multiple columns
top_students = df.sort_values(by=['GPA', 'Study_Hours_Per_Week'], ascending=[False, False])
print(top_students.head(5))
```

Q12. Rename the column 'Screen_Time_Hours_Per_Day' to just 'Daily_Screen_Time' and save the change to the same DataFrame.

```
# Renaming columns inplace
df.rename(columns={'Screen_Time_Hours_Per_Day': 'Daily_Screen_Time'}, inplace=True)
print(df.columns)
```

Q13. Check if there are any duplicate records based on the 'Student_ID' column and count them.

```
# Checking for duplicate IDs
duplicate_count = df.duplicated(subset=['Student_ID']).sum()
print(f"Number of duplicate student records: {duplicate_count}")
```

Part 5: Advanced Grouping & Categorization (Pandas)

Q14. Create a new column called 'Sleep_Status'. If a student gets 7 or more hours of sleep, label them 'Rested'. Otherwise, label them 'Sleep_Deprived'. (Hint: Use np.where)

```
# Conditional classification using NumPy where
df['Sleep_Status'] = np.where(df['Sleep_Hours_Per_Night'] >= 7.0, 'Rested', 'Sleep_Deprived')
print(df[['Sleep_Hours_Per_Night', 'Sleep_Status']].head(10))
```

Q15. Find the maximum and minimum GPA for each unique major using the .agg() function.

```
# Custom aggregations per group
gpa_range_by_major = df.groupby('Major')['GPA'].agg(['min', 'max'])
print(gpa_range_by_major)
```

Q16. Calculate the average coffee consumption of students, grouped by both their Major AND whether they have a Part-Time Job.

```
# Grouping by multiple categorical variables
multi_group = df.groupby(['Major', 'Part_Time_Job'])['Coffee_Cups_Per_Week'].mean()
print(multi_group)
```

Python NumPy & Pandas: Extended Practice Assignment

Dataset: Student Life & Daily Habits (student_life_dataset_200.csv)

Part 6: Array Manipulations & Conditionals (NumPy)

Q17. Convert the 'Study_Hours_Per_Week' column into a NumPy array, and use a NumPy boolean mask to find the exact number of students who study less than 10 hours a week.

```
# Convert column to array
study_array = df['Study_Hours_Per_Week'].to_numpy()

# Create a boolean mask and sum True values
less_than_10 = study_array < 10
count_low_study = np.sum(less_than_10)
print(f"Students studying < 10 hours: {count_low_study}")
```

Q18. Find the standard deviation and variance of the students' GPA array using NumPy functions.

```
# Extract GPA array
gpa_array = df['GPA'].to_numpy()

# Compute standard deviation and variance
gpa_std = np.std(gpa_array)
gpa_var = np.var(gpa_array)
print(f"GPA Standard Deviation: {gpa_std:.2f}")
print(f"GPA Variance: {gpa_var:.2f}")
```

Q19. Use NumPy to find the unique names of all the majors present in the dataset without using the Pandas unique function.

```
# Extract Major column as array
majors_array = df['Major'].to_numpy()

# Use np.unique
unique_majors = np.unique(majors_array)
print("Unique majors identified by NumPy:")
print(unique_majors)
```

Q20. Suppose every student's GPA is given a 'curve' increase of 5%. Multiply the entire GPA NumPy array by 1.05 and display the first 5 adjusted GPAs.

```
# NumPy vector broadcasting (multiplying the whole array at once)
curved_gpas = gpa_array * 1.05
print("First 5 original GPAs: ", gpa_array[:5])
print("First 5 curved GPAs:   ", curved_gpas[:5].round(2))
```